

# Backtracking

## 回溯法

本页面将简要介绍回溯法的概念和应用。

## 简介

回溯法是一种经常被用在 [深度优先搜索 \(DFS\)](#) 和 [广度优先搜索 \(BFS\)](#) 的技巧。

其本质是：走不通就回头。

## 过程

- 1. 构造空间树；
- 2. 进行遍历；
- 3. 如遇到边界条件，即不再向下搜索，转而搜索另一条链；
- 4. 达到目标条件，输出结果。

## 例题

### ▼ [USACO 1.5.4 Checker Challenge](#)

现在有一个如下的 的跳棋棋盘，有六个棋子被放置在棋盘上，使得每行，每列，每条对角线（包括两条主对角线的所有对角线）上都至多有一个棋子。

|    |       |   |   |   |   |   |   |
|----|-------|---|---|---|---|---|---|
| 1  | 0     | 1 | 2 | 3 | 4 | 5 | 6 |
| 2  | ----- |   |   |   |   |   |   |
| 3  | 1     |   |   | O |   |   |   |
| 4  | ----- |   |   |   |   |   |   |
| 5  | 2     |   |   |   | O |   |   |
| 6  | ----- |   |   |   |   |   |   |
| 7  | 3     |   |   |   |   |   | O |
| 8  | ----- |   |   |   |   |   |   |
| 9  | 4     |   | O |   |   |   |   |
| 10 | ----- |   |   |   |   |   |   |
| 11 | 5     |   |   |   | O |   |   |
| 12 | ----- |   |   |   |   |   |   |
| 13 | 6     |   |   |   |   | O |   |
| 14 | ----- |   |   |   |   |   |   |

上面的布局可以用序列 来描述，第 个数字表示在第 行的第 列有一个棋子，如下所示

行号：

列号：

这只是跳棋放置的一个方案。请编一个程序找出所有方案并把它们以上面的序列化方法输出，按字典顺序排列。你只需输出前 个解并在最后一行输出解的总个数。特别注意：你需要优化你的程序以保证在更大棋盘尺寸下的程序效率。

► [参考代码](#)

## ▼ [迷宫](#)

现有一个尺寸为  $n \times m$  的迷宫，迷宫里有  $k$  处障碍，障碍处不可通过。给定起点坐标和终点坐标，且每个方格最多经过一次，问有多少种从起点坐标到终点坐标的方案。在迷宫中移动有上、下、左、右四种移动方式，每次只能移动一个方格。数据保证起点上没有障碍。

### ► [参考代码](#)

---

本页面最近更新：2023/2/18 07:57:07，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[NachtgeistW](#), [Alisahhh](#), [FFjet](#), [iamtwz](#), [lr1d](#), [kenlig](#), [ksyx](#), [luoguyuntianming](#), [miaotony](#), [partychicken](#), [sshwy](#), [StudyingFather](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用